

emme

Health-Plan Onboarding — demo brief

TOA Health Hack · Track 2 · med-hack-template.vercel.app

The 3-minute run

Two people max at the front. One drives the phone, one talks. Do not both talk. Mirror the phone if possible; if not, hold it up and narrate.

0:00 – 0:25 TALKER — landing page on screen

"Emme tells you what your care will cost before you get the bill. But every calculation needs your plan details, and there was no clean way to collect them. The hard part isn't the data model — it's that people approach healthcare forms with anxiety and learned helplessness. A form that feels like a medical form kills trust before you earn it." Point at the split panel: raw EOB on the left, plain English on the right. "That's the problem in one screen."

0:25 – 0:50 DRIVER — tap Start form, upload the PDF

"You start by uploading your Summary of Benefits or a recent statement — or skipping it entirely, which is equally prominent." Upload sample-eob-synthetic.pdf. Twelve fields populate.

0:50 – 1:05 BOTH — let the counter land, then talk

"Watch the question count." It drops from 21 to 13. PAUSE for two seconds before speaking again. "Every field it filled is marked 'from your document' and stays editable. We never silently decide for you."

1:05 – 1:30 DRIVER — scroll one section, tap a skip

"Every question explains why we need it, inline — not behind a tooltip, because on a phone there's no hover." Tap 'I don't have this' on any field. "Nothing is required. Skipping is a first-class path, not a failure."

1:30 – 1:45 DRIVER — reload the page

Pull down and reload mid-flow. "Everything's still there. No account, no password — it saves as you go, because people get interrupted." This beat is cheap and it proves the requirement instead of claiming it.

1:45 – 2:15 TALKER — the summary screen

"And here's the payoff. Most people have never had their own plan explained back to them." Read one line out loud: "You've paid \$840 of your \$2,500 deductible. After another \$1,660, your plan starts covering 80%." Then: "That's their own numbers, in words. We don't project the cost of care — that's Emme's engine, and it's downstream of us."

2:15 – 2:35 DRIVER — tap 'Show me the raw data'

"And it exports as clean JSON, ready for their cost engine. Numbers as numbers, camelCase keys, plus metadata on which fields came from a document and which were skipped."

2:35 – 3:00 TALKER — close

"One schema file drives the form, the tooltips, the extraction target, the summary and the export — so adding a carrier never touches the UI. Next we'd add input validation depth and split the cost-sharing section, which is where we'd expect drop-off. We'd measure that rather than guess."

Know your own code

So nobody gets caught flat. Any one of you should be able to say these.

The one-sentence architecture

lib/schema.ts is the single source of truth — 35 fields across 7 sections, each carrying its own label, type, plain-language 'why' copy and a hint for where to find it on a statement. The form UI, the tooltips, the extraction target, the summary screen and the JSON export are all derived from it. Add a field there and it appears everywhere.

That's the answer to 'how does this scale?'. It's also why four people could build it in four hours without colliding.

The pieces

- lib/schema.ts — 35 fields. The 'why' string is a required property on the type, so a field literally cannot ship without justifying itself to the member.
- lib/store.tsx — React context over localStorage (key emme:intake:v1, 400ms debounce). Tracks per-field provenance: typed, extracted, or inferred. A value you type always beats one we extracted.
- lib/extract/heuristic.ts — regex plus label-proximity search over the PDF text layer. Scores a confidence per field based on how close the value sits to its label.
- lib/extract/pdf-text.ts — our own PDF text reader, built on node:zlib alone. Inflates content streams and pulls string literals out of the Tj/TJ text operators.
- lib/summarize.ts — turns answers into plain-English sentences about the member's own plan. Deliberately does no cost projection.
- app/api/extract/route.ts — always returns 200. A failed parse is a normal path that falls through to manual entry, never an error the member has to handle.

Stack

Next.js 16 App Router, React 19, TypeScript, Tailwind v4, shadcn/ui, deployed on Vercel. No database, no auth, no server-side storage.

Who built what

- Aaryan — schema, state, intake flow, summary screen, landing page
- Shreyas — document extraction: the heuristic parser, LLM dispatch and the API route
- Copy and client research — the 'why' strings and the Emme conversation
- Pitch, demo data and testing

Numbers worth remembering

- 35 fields, 7 sections, 21 questions asked cold (the rest are deferred or conditional)
- 12 fields extracted from the sample document; question count drops 21 to 13
- 9 fields are conditionally hidden — no HSA questions unless you have one

Questions they will ask

Is that actually parsing the document, or is it hardcoded?

It's parsing. Regex and label-proximity matching over the PDF text layer — no API key, no vision model. We wrote our own text reader because pdf-parse depends on pdfjs, which needs DOM globals that don't exist in a serverless function. It worked locally and 500'd in production, so we replaced that layer with about eighty lines on top of node:zlib.

Is that a real Explanation of Benefits?

No, and deliberately so. It's synthetic, generated by a script in our repo, and it says so on its face. The sample the challenge linked to is an annotated walkthrough with no dollar figures in it, so we authored a realistic one. We're not putting real patient documents through a hackathon prototype.

What about HIPAA and PHI?

Nothing we built handles real PHI today, and that's a choice. In production this needs a BAA with any model provider, encryption at rest, and a hard rule that no document contents go into a third-party prompt without that agreement in place. Right now data never leaves the browser except for the parse call, and we store nothing server-side.

Where is the data actually stored?

localStorage in the browser. No account, no database. That's what makes 'save and resume' work with zero friction, and it means we're not holding health data we don't need. For production it would move behind auth, but the export shape wouldn't change.

How accurate is the extraction?

On a well-structured statement, twelve of the fields we care about. On a scan or a photo with no text layer, nothing — and that's fine, because it falls through to manual entry with an explanation rather than failing. Every extracted value is labelled and editable, so the member is the final check. We'd rather be honestly partial than confidently wrong about someone's deductible.

Why don't you show what care will cost?

We asked Emme, and it's out of scope on purpose. We collect the inputs; their engine does the pricing. We do explain the member's own plan back to them in plain English, which needs no pricing data and is the part nobody had done for them before.

How does this scale to fifty carriers?

The form is generated from one schema file. Adding a carrier, a field, or a new tooltip is a data change, not a UI change. The extraction layer is behind a single interface — heuristic today, an LLM path already stubbed behind an environment variable, and a carrier-specific parser could slot in the same way.

Harder questions

What's broken or unfinished?

Validation is shallow — we check format, not plausibility. The cost-sharing section is ten questions on one screen, which is where we'd expect people to drop off. And we've tested the flow, not real members. Those are the three things we'd fix first, in that order.

What would you do with another week?

Split the cost-sharing screen and measure the drop-off. Add carrier-specific extraction templates, since a BCBS statement and an Aetna one differ in predictable ways. And run it in front of ten actual members, because every piece of copy in there is currently our guess about what reassures people.

Why Next.js when the pre-hack notes said Python?

The Python boilerplate was for the other track. This one is a user-facing product and design challenge, so we went with the stack that gets a mobile-first flow deployed fastest. The extraction is the only backend piece and it's a single API route.

How did four people not step on each other?

Two people on code, two on copy and pitch — with strict file ownership written into the repo before anyone started. One person owned the app, one owned extraction, and they touched disjoint directories. One merge, one conflict, resolved in two minutes.

Is this production ready?

No, and I wouldn't claim it. It's a working end-to-end prototype with real extraction and an honest export format. What's missing before real members touch it: auth, a real datastore, a BAA if an LLM path is enabled, accessibility testing with actual assistive tech, and validation that catches implausible values rather than just malformed ones.

What if someone enters the wrong numbers?

We validate format on blur and explain the problem in plain language, but we never block. The summary screen shows every answer back for review before export, and flags what's missing along with what each gap would have sharpened. Beyond that it's their plan and their data — we'd rather show our working than lock them out.

If the demo breaks

- Backup screen recording — have it on the phone before you walk up.
- The manual path works with no network calls at all. Skip the upload and keep going.
- Say it plainly and move on. Judges have seen a hundred demos fail; how you recover is the signal.

All figures in the demo are synthetic. No real patient data appears anywhere in this prototype.